

bi-toolkit User Guide

Dan Powers / Quandary Labs

June 24, 2026

CONTENTS

Welcome	
Who This Is For	
What The Toolkit Does	
Trustworthy Analysis	
Two Kinds Of Work	
ANALYSIS	
DATA PRODUCT	
What It Feels Like	
Your Role	
Setup	
What Claude Code Is	
Install Or Open Claude Code	
What A Project Folder Is	
Create An Empty Folder	
Open Claude Code In That Folder	
What To Tell Claude First	
Install	
The One-Liner	
What Lands In The Folder	
Check The Install	
First Claude Command	

First Run

Before You Start

Step 0: Intake

Step 1: Frame

Step 2: Spec

Step 5: Discovery

Step 6: Execute

Step 7: Validate

Step 9: Deliver

What A Block Looks Like

Workflow

Diagram In Text

The Stages

When To Use ANALYSIS

When To Use DATA PRODUCT

What To Type

Gates And Receipts

Why Gates Exist

Receipt Files

questioncheck

validationcheck

govcheck

datacheck

reprocheck

See Status

Skills

Router

Intake And Framing

Analysis Techniques

Validation And Review

Communication

Documentation

Production And Output

How To Ask For A Skill

Agents And Models

The Simple Rule

Main Model

The 4 Sub-Agents

Why Use Sub-Agents

The Model Dial

What You Will See

Reports And Dashboards

Report Engine

Report Contents

PDF Requirements

Dashboard Engine

Dashboard Panel Types	
Dashboard Required Fields	
Branding	
Recurring And Monitoring	
What Is Verified In This Repo	
Versioning	
What Changed Since Last Run	
Drift And Refresh Monitoring	
Distribution Handoff	
Living Metric Store	
Recurring Report Checklist	
Conductor And Help	
Active Analysis	
Every-Turn Reminder	
/assay help	
/assay status	
/assay finish	
/assay resume	
/assay ledger	
New User Guidance	
Troubleshooting FAQ	
It stopped and asked for a spec. Did I do something wrong?	
What is a receipt?	

The PDF did not generate. Is the report lost?

I closed the window. How do I pick up?

It says source-of-truth is not configured.

The numbers need a source of truth, but we have two.

It found a new methodology fork during execution.

It says data classification is unknown.

It says a governing doc changed.

It says reproducibility is unverified.

Claude is asking too many questions.

Claude is making assumptions.

Tips For Working With An AI Assistant

Glossary

A

B

C

D

E

F

G

H

I

J

K

L	
M	
N	
O	
P	
Q	
R	
S	
T	
U	
V	
W	
Z	

Welcome

bi-toolkit is a Claude Code (an AI assistant you run in a folder) kit for BI (business reporting and metrics) work. It gives you a repeatable path for answering data questions and building recurring reports.

The workflow is called the assay loop. Assay means test quality and composition. In this kit, that means every important number is defined, checked, and explained before it is delivered.

[Index](#) | Next: Setup

Who This Is For

Use this guide if you:

- build reports, dashboards, scorecards, or analysis;
- know your data and business process;
- are new to AI (software that drafts and checks work);
- are new to Claude Code (an AI assistant you run in a folder);
- want a patient, step-by-step process.

You do not need to be a developer. You will type short commands and answer plain questions.

What The Toolkit Does

The toolkit helps Claude do the legwork:

- collect context;
- define the question;
- find method choices before numbers run;
- run or guide the analysis;
- reconcile results to source-of-truth (official place to compare);
- write receipts (saved proof files);

- package a report or dashboard.

You make the judgment calls:

- what decision matters;
- which metric definition is right;
- what source-of-truth is official;
- whether a caveat changes the decision;
- whether an accepted risk is reasonable.

Trustworthy Analysis

Trustworthy analysis means a person can answer five questions:

1. What question did we answer?
2. How did we define each metric?
3. What data did we use?
4. Did the result reconcile to source-of-truth (official place to compare)?
5. What limits or caveats affect the decision?

The kit blocks delivery when proof is missing. A block is a required stop, not a crash.

Two Kinds Of Work

ANALYSIS

ANALYSIS means a one-time answer. Use it when someone asks:

- "Why did renewal revenue drop?"
- "Which stores missed margin?"
- "Did the test improve conversion?"
- "What changed since last quarter?"

The output is a validated answer with evidence, caveats (limits that affect trust), and next steps.

DATA PRODUCT

DATA PRODUCT means a recurring report or dashboard. Use it when you need:

- a weekly scorecard;
- a monthly finance dashboard;
- a recurring operations exception report;
- a report that refreshes on a schedule.

The output is a recurring report or dashboard with stricter validation. A wrong recurring number repeats until someone catches it, so the kit treats it more carefully.

What It Feels Like

You type:

```
/assay frame
```

Claude responds with questions such as:

```
What decision will this answer support?  
Is this a one-time analysis or a recurring data product?  
What source-of-truth should the final number tie to?
```

You answer in normal words:

```
This is a one-time analysis. Finance wants to know why Q2 renewal revenue  
dropped before the forecast meeting. Renewal revenue should tie to the  
finance close report.
```

Claude then guides the next step. You do not have to remember the whole workflow.

Your Role

Think of Claude as a careful analyst assistant. It can draft, inspect, compare, and summarize. It cannot decide what your business should count as official.

Good operator input:

Use invoice date, not close date, because Finance reports renewal revenue by invoice month.

Weak operator input:

Just use the normal date.

The first answer gives the kit a ruling (operator-approved method choice). The second answer hides a method choice that could change the number.

Setup

This page gets your computer ready at a beginner level. You only need a project folder (folder for one work area), Claude Code (an AI assistant you run in a folder), and a terminal (the text command window).

Previous: Welcome | [Index](#) | Next: Install

What Claude Code Is

Claude Code is an AI assistant you run inside a folder. It can read files, write files, run commands, and ask you questions.

For this toolkit, Claude Code is the place where you type:

```
/assay help
```

and Claude walks you through the BI workflow.

Install Or Open Claude Code

Claude Code installation can differ by company account and computer setup. Use your company instructions or Anthropic's current Claude Code instructions if Claude Code is not installed yet.

After it is installed, you should be able to open a terminal (the text command window) and type:

```
claude
```

You will usually see a Claude Code screen or prompt. It may ask you to sign in. Sign in with the account your company expects you to use.

If `claude` is not found, you may see:

```
command not found: claude
```

That means Claude Code is not installed or not on your PATH (the command lookup list). Ask your IT or AI tools owner for the correct install steps.

What A Project Folder Is

A project folder is one folder that holds one body of work. For this kit, make a fresh folder for each BI project or reporting area.

Examples:

```
renewal-analysis/  
finance-scorecard/  
store-ops-dashboard/
```

Do not install the kit into your Downloads folder or your whole Documents folder. Use a small, empty folder.

Create An Empty Folder

Open your terminal (the text command window). Type:

```
mkdir renewal-analysis  
cd renewal-analysis
```

What you will see:

No output usually means it worked. `mkdir` means make directory, which is another word for folder. `cd` means change directory, which moves the terminal into that folder.

Check where you are:

```
pwd
```

You will see something like:

```
/Users/you/renewal-analysis
```

Check that the folder is empty:

```
ls
```

You should see no files, or only files you intentionally put there.

Open Claude Code In That Folder

From the same folder, type:

```
claude
```

You are now using Claude Code inside this project folder. When you install bi-toolkit, it will place its files in this folder only.

What To Tell Claude First

If you are new, say:

```
I am new to Claude Code. I am setting up bi-toolkit for a BI project.  
Please go step by step.
```

Then continue to Install.

Install

Install bi-toolkit from a fresh project folder (folder for one work area). The installer copies the kit into that folder and leaves your other folders alone.

Previous: Setup | [Index](#) | Next: First Run

The One-Liner

In the empty folder you made, type this in the terminal (the text command window):

```
curl -fsSL https://raw.githubusercontent.com/dep0we/bi-  
toolkit/main/bootstrap.sh | bash
```

What this means:

- `curl` downloads a file from the internet.
- `bash` runs a shell script (small command program).
- `|` pipes output, meaning passes it along.

What you will see:

```
Downloading bi-toolkit...
Installing into /path/to/your/project...
Installing bi-toolkit assay spine into: /path/to/your/project
  installed: .claude/skills/ (... skills, incl. the /assay router)
  installed: .claude/hooks/
  installed: .claude/workflows/...
  created: assay.config.jsonc
  created: CLAUDE.md
  created: PLAYBOOK.md
  created: methodology.md
  created: model-dial.md
  created: claude-md-guide.md
  created: data-safety.md
  seeded: seed-memory/

Done. Next step: run /assay help, then /assay intake.
Receipts (saved proof files) will live under .assay/receipts/.

bi-toolkit is installed.
Open Claude Code in this folder and run: /assay help
When you are ready to set up the project, run: /assay intake
```

The exact list may be longer, but those are the important parts.

What Lands In The Folder

Plain map:


```

.claude/
  skills/      skills Claude can use
  agents/      worker agents for legwork
  workflows/    scripts that run gates and render outputs
  hooks/       reminder shown each turn

assay.config.jsonc  project settings
CLAUDE.md           project memory and rules
PLAYBOOK.md        lifecycle manual
methodology.md      why the method works
model-dial.md        when to use each model
data-safety.md       sensitive-data rules
seed-memory/         reusable BI lessons
.gitignore           ignores runtime proof folders

```

JSONC means JSON with comments. JSON means structured key-value data.

The installer is rerunnable. It overwrites kit tooling, but it does not overwrite your existing `assay.config.jsonc`, `CLAUDE.md`, or docs.

Check The Install

Type:

```
bash .claude/workflows/assay-help.sh
```

You should see:

```

assay help

What this kit does: it guides BI work from question to trusted answer.
...
Current next step:
next required step: /assay intake
No active analysis or receipts were found. Receipts are saved proof
files.

```

First Claude Command

Open Claude Code in the folder if it is not already open:

```
claude
```

Then type this inside Claude Code:

```
/assay intake
```

Claude will start asking about:

- warehouse (central store for analysis data);
- BI tool (reporting or dashboard system);
- query language (how data is requested);
- source-of-truth (official place to compare);
- validation habit (how numbers are checked);
- stakeholders (people who use or approve);
- delivery rules (what done means);
- sensitive data (data needing special handling).

You do not need perfect answers. If you are unsure, say so. The kit is designed to ask follow-up questions.

First Run

This page walks through one complete ANALYSIS (one-time answer) from intake to delivered report. The example question is: “Why did Q2 renewal revenue drop?”

Previous: Install | [Index](#) | Next: Workflow

Before You Start

You are in Claude Code (an AI assistant you run in a folder). You have installed bi-toolkit in a project folder (folder for one work area).

Use a stable analysis id (short saved-work name):

```
renewal-revenue-q2
```

Step 0: Intake

Type:

```
/assay intake
```

Claude asks:

```
What warehouse, BI tool, and query language does this project use?  
What are the source-of-truth systems for key metrics?  
How do you validate numbers today?  
Who are the decision makers, reviewers, and audience?  
What counts as high-stakes work here?  
What delivery rules should every answer follow?
```

You answer:

Warehouse: Snowflake. BI tool: Tableau. Query language: SQL.
 Renewal revenue source-of-truth: Finance close report.
 Validation habit: totals must tie to Finance within \$100 or the variance must be explained.
 Decision maker: VP Sales. Reviewer: Finance analyst. Audience: Sales leadership.
 High-stakes: revenue, forecast, headcount, and board metrics.
 Done means a plain report with caveats and source notes.

What happens:

- Claude drafts updates to `assay.config.jsonc`.
- Claude drafts updates to `CLAUDE.md`.
- Claude should ask before writing those project settings.

Step 1: Frame

Type:

```
/assay frame
```

Claude asks:

Is this ANALYSIS or DATA PRODUCT?
 What decision will this answer support?

You answer:

This is ANALYSIS. It is a one-time answer for why Q2 renewal revenue dropped. The decision is whether Sales should focus recovery work on a segment before the forecast meeting.
 Use analysis id renewal-revenue-q2.

What happens:

```
assay-active-set:renewal-revenue-q2:analysis
```

Active means saved work to resume first.

Step 2: Spec

Type:

```
/assay spec renewal-revenue-q2
```

Claude asks:

```
What exact question are we answering?
Which metrics need definitions?
What is in scope and out of scope?
What would count as a valid answer?
What decision impact should be recorded?
```

You answer:

```
Question: Why did Q2 renewal revenue drop compared with Q1?
Metric: renewal revenue means invoiced renewal ARR in the quarter,
excluding new-logo revenue and one-time services.
Scope: Q1 and Q2 of this fiscal year, all active customer accounts,
grouped by segment and sales owner.
Valid answer: quantify the drop, identify the largest drivers, reconcile
total renewal revenue to Finance, and state caveats.
Decision impact: affects forecast and Sales recovery focus, so it drives
money and strategy.
Data classification: internal aggregate unless customer-level tables are
exported.
```

Claude writes a spec receipt (saved proof file):

```
assay-receipt-written:spec:.assay/receipts/renewal-revenue-q2-spec-
receipt.json
```

Why this matters: `questioncheck` will not let execution start without this receipt.

Step 5: Discovery

Type:

```
/assay discovery renewal-revenue-q2
```

Discovery means finding method choices before results. Claude does not compute final numbers here.

First, the kit snapshots governing docs (protected rule files):

```
assay-gate-ok:govcheck-snapshot
```

Then Claude lists methodology forks (choices that change numbers), such as:

```
Fork: Which date counts?
Option A: invoice date. Consequence: matches Finance close reporting.
Option B: contract close date. Consequence: may shift revenue into a
different quarter.
Recommendation: invoice date, because the source-of-truth is Finance
close.

Fork: Gross or net renewal revenue?
Option A: gross renewals only. Consequence: shows booked renewal volume.
Option B: net of credits and cancellations. Consequence: ties better to
actual revenue impact.
Recommendation: use the Finance definition named in the close report.
```

You make rulings:

```
Use invoice date. Use the Finance close definition. Exclude one-time
services. Group by Finance segment, not Sales segment, because Finance
owns the source-of-truth.
```

Claude records the rulings (operator-approved method choices):

```
assay-discovery-recorded:.assay/rulings/renewal-revenue-q2-latest-
discovery.json
assay-rulings-written:.assay/rulings/renewal-revenue-q2-rulings.json
```

Step 6: Execute

Type:

```
/assay execute renewal-revenue-q2
```

The front gate runs:

```
assay-gate-ok:questioncheck
assay-gate-ok:rulingscheck
```

Claude delegates legwork:

- `eda-profiler` checks tables, freshness, nulls (blank or unknown values), duplicates, and outliers (unusual values that can skew).
- `query-runner` runs the ruled queries exactly to the spec.

Claude may say:

```
I found a new Tier-A fork: customer segment differs between Finance and
Sales. This can change the driver table. Please rule which segment source
to use before I continue.
```

That is a block. It is the system protecting you from a silent choice. Answer with a ruling:

```
Use Finance segment for the main answer. Add Sales segment only as a
supporting cut if needed.
```

Step 7: Validate

Type:

```
/assay validate renewal-revenue-q2
```

Claude sends the `reconciler` sub-agent (worker agent with narrow task) to tie results to source-of-truth (official place to compare). It sends a fresh `red-teamer` sub-agent to attack the conclusion.

You may see:

```
Finance close total: $4,820,000  
Analysis total: $4,819,944  
Variance: $56  
Status: matched within tolerance
```

Claude writes:

```
assay-receipt-written:validation:.assay/receipts/renewal-revenue-q2-  
validation-receipt.json  
assay-receipt-written:adversarial-review:.assay/receipts/renewal-revenue-  
q2-adversarial-review-receipt.json
```

The review receipt includes scores:

```
confidence: 4  
dataCompleteness: 4  
methodologySoundness: 4  
reproducibility: 3
```

Step 9: Deliver

Type:

```
/assay deliver renewal-revenue-q2
```

Delivery preflight runs:


```
assay-gate-ok:validationcheck  
assay-gate-ok:govcheck  
assay-gate-ok:datacheck  
assay-gate-ok:reprocheck
```

If PDF tools are installed, the report renderer writes HTML and PDF. If not, it still writes HTML.

What you may see:

```
assay-report-html:.assay/deliverables/renewal-revenue-q2/report-  
20260624T153000Z.html  
assay-report-pdf-note:No PDF renderer, meaning a tool that makes PDF  
files, was available. Open the HTML report and use print-to-PDF if a PDF  
is needed.  
assay-report-receipt:.assay/receipts/renewal-revenue-q2-deliverable-  
receipt.json
```

The report includes:

- conclusion;
- key findings;
- evidence;
- methodology (chosen analysis approach);
- caveats (limits that affect trust);
- reconciliation notes;
- score;
- next steps.

What A Block Looks Like

Example:

```
assay-gate-failed:missing-spec  
questioncheck: Stage 6 is blocked because no Stage 2 spec receipt exists.  
Create .assay/receipts/renewal-revenue-q2-spec-receipt.json with the  
question, metric definitions, and what a valid answer looks like. A  
receipt is a saved proof file.
```

This is not an error in your work. It means the kit will not let numbers run before the question and metric definitions are written down.

What to type:

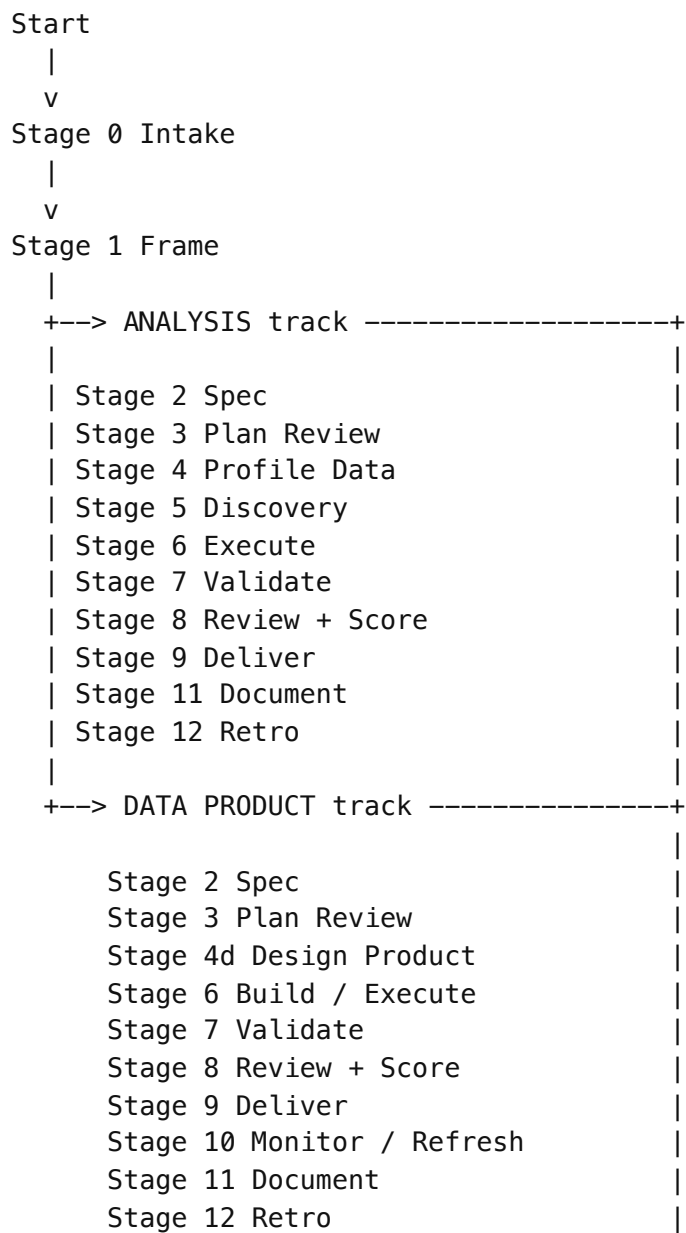
```
/assay spec renewal-revenue-q2
```

Workflow

The assay loop is the lifecycle (ordered path from question to delivery). It has one shared spine and two tracks: ANALYSIS (one-time answer) and DATA PRODUCT (recurring report or dashboard).

Previous: First Run | [Index](#) | Next: Gates And Receipts

Diagram In Text



The Stages

Stage 0 Intake captures your BI stack (tools and systems), source-of-truth (official place to compare), validation habit (how numbers are checked), stakeholders, and done rules.

Stage 1 Frame decides whether the work is ANALYSIS or DATA PRODUCT and names the decision the answer supports.

Stage 2 Spec defines the question, metric definitions (exact calculation rules), scope, and valid answer. It writes the spec receipt (saved proof file).

Stage 3 Plan Review pressure-tests decision value, data availability, methodology (chosen analysis approach), and plain language.

Stage 4 Profile Data checks shape, freshness, missing values, duplicates, outliers (unusual values that can skew), and table relationships.

Stage 4d Design Product is for DATA PRODUCT work. It chooses layout, metrics, refresh cadence (how often it updates), access, and semantic layer (shared metric definitions).

Stage 5 Discovery finds methodology forks (choices that change numbers) before results are computed.

Stage 6 Execute runs the ruled analysis or builds the report/dashboard.

Stage 7 Validate reconciles results to source-of-truth (official place to compare).

Stage 8 Review + Score uses a fresh red-teamer (independent checking agent) to attack weak spots and score the work.

Stage 9 Deliver packages the answer, evidence, caveats (limits that affect trust), and next steps.

Stage 10 Monitor / Refresh is for DATA PRODUCT work. It checks recurring runs and metric drift (unexpected number movement).

Stage 11 Document records assumptions, queries, decisions, and validation notes.

Stage 12 Retro captures lessons for the next analysis.

When To Use ANALYSIS

Use ANALYSIS when:

- the question is one-time;
- the audience needs an answer, not a maintained dashboard;
- the time window is specific;
- the output can be a report, table, or recommendation.

Examples:

```
Why did Q2 renewal revenue drop?  
Which locations drove labor overage last month?  
Did the campaign change conversion?
```

When To Use DATA PRODUCT

Use DATA PRODUCT when:

- the report will recur;
- people will act from the same numbers repeatedly;
- refresh timing matters;
- ownership, access, and layout matter;
- a wrong number could repeat.

Examples:

```
Build a weekly renewal risk dashboard.  
Create a monthly gross margin scorecard.  
Replace a manual executive report.
```

What To Type

If you know the work is one-time:

```
/assay frame
```

Then say:

This is ANALYSIS because it answers one question for one decision.

If you know the work is recurring:

/assay frame

Then say:

This is DATA PRODUCT because it will refresh every week and be used repeatedly.

If you are unsure:

/assay help

You will see the current next step and plain guidance.

Gates And Receipts

A gate is a required stop-check before continuing. A receipt is a saved proof file. Gates read receipts and other project state. If proof is missing, the gate blocks.

Previous: Workflow | [Index](#) | Next: Skills

Why Gates Exist

The gates protect you from common BI failures:

- running numbers before defining the metric;
- delivering results that do not tie to source-of-truth (official place to compare);
- sharing sensitive data without a recorded audience;
- changing governing rules during the work;
- shipping a recurring report that cannot be rerun.

A gate failure is not a scolding. It is a checklist item with a clear fix.

Receipt Files

Receipts usually live here:

```
.assay/receipts/
```

Common receipts:

```
<analysis-id>-spec-receipt.json  
<analysis-id>-validation-receipt.json  
<analysis-id>-adversarial-review-receipt.json  
<analysis-id>-data-safety-receipt.json  
<analysis-id>-deliverable-receipt.json  
<analysis-id>-govbaseline.json
```


JSON means structured key-value data. You normally do not hand-edit receipts. Claude writes them through `.claude/workflows/receipt.sh`.

questioncheck

Protects: Stage 6 Execute.

What it checks:

- a Stage 2 spec receipt exists;
- the spec receipt has `kind: "spec"` or `kind: "trivial"`;
- a real spec has a question, metric definitions, and valid answer;
- a trivial spec has a one-line reason.

Common block:

```
assay-gate-failed:missing-spec
questioncheck: Stage 6 is blocked because no Stage 2 spec receipt
exists...
```

How to satisfy it:

```
/assay spec <analysis-id>
```

Give Claude:

```
Question:
Metric definitions:
Scope:
Valid answer:
Decision impact:
Track: analysis or data-product
```

validationcheck

Protects: Stage 9 Deliver.

What it checks:

- validation receipt exists;
- validation receipt says `reconciled: true`;
- reconciliation details exist;
- non-trivial work has an adversarial-review receipt;
- the review has scores for confidence, data completeness, methodology soundness, and reproducibility;
- high-stakes work and data products have configured source-of-truth.

Common blocks:

```
assay-gate-failed:missing-validation
validationcheck: Stage 9 is blocked because no Stage 7 validation receipt
exists.
```

```
assay-gate-failed:sub-threshold-score
validationcheck: A Stage 8 score is below threshold...
```

```
assay-gate-failed:source-of-truth-unconfigured
validationcheck: This work is high-stakes... but sourceOfTruth is not
configured.
```

How to satisfy it:

```
/assay validate <analysis-id>
```

If source-of-truth is missing:

```
/assay intake
```

Then add the official source for each key metric.

govcheck

Protects: Stage 9 Deliver.

What it checks:

- discovery created a governing-doc baseline (saved starting copy);
- protected rule files did not change during the analysis.

Default protected files come from `assay.config.jsonc`:

```
CLAUDE.md
methodology.md
docs/DECISIONS.md
docs/spec/assay-spec.md
```

Common block:

```
assay-gate-failed:governing-doc-edit
govcheck: Delivery is blocked because a guarded governing doc changed
during this analysis...
```

How to satisfy it:

- If the change was accidental, restore or undo the rule-file change.
- If a person approved the rule update, resnapshot with explicit approval:

```
ASSAY_GOVCHECK_APPROVED=1 bash .claude/workflows/govcheck.sh resnapshot
<analysis-id>
```

Only do that when the rule change is intentional.

datacheck

Protects: Stage 9 Deliver.

What it checks:

- data classification (how sensitive data is) is known;
- sensitive data has a data-safety receipt;
- the receipt has audience, export destination, detail level, and operator sign-off;
- data leaving the company goes only to an approved destination.

Sensitive data means personal, health, payroll, or customer records.

Common blocks:

```
assay-gate-failed:unknown-classification
datacheck: Delivery is blocked because the data classification is
unset...
```

```
assay-gate-failed:missing-data-safety
datacheck: Delivery is blocked because this work touches sensitive
data...
```

How to satisfy it:

Tell Claude:

```
Data classification: customer records.
Delivery audience: internal finance leadership.
Data leaves company: false.
Export destination: none.
Detail level: aggregate.
Operator sign-off: I approve this audience and handling.
```

Claude writes the data-safety receipt.

reprocheck

Protects: Stage 9 Deliver.

What it checks:

- if `reproCommand` is set in `assay.config.jsonc`, the command exits zero;

- zero means success;
- non-zero means the rerun failed or changed beyond tolerance.

If `reproCommand` is not set, it does not block, but it warns:

```
reprocheck: NOTE - reproducibility unverified because reproCommand is
unset.
assay-gate-ok: reprocheck
```

For DATA PRODUCT work, it also warns that recurring reports should set `reproCommand`.

How to satisfy it:

Add a command in `assay.config.jsonc` that reruns the analysis or report build:

```
"reproCommand": "bash scripts/build-renewal-report.sh"
```

Then run:

```
/assay deliver <analysis-id>
```

See Status

Type:

```
/assay status
```

You will see:

completed stages:

- Stage 2 spec receipt

open findings:

- missing-validation [validationcheck]: Missing Stage 7 validation receipt...

blocking gate: validationcheck (missing-validation)

next required step: /assay validate renewal-revenue-q2

Skills

A skill is a reusable instruction set Claude can use for a specific kind of BI work. The kit includes the `/assay` router plus 31 domain skills.

Previous: Gates And Receipts | [Index](#) | Next: Agents And Models

Router

Skill	Plain description	Use this when...
<code>assay</code>	Runs the BI quality loop and commands.	You type <code>/assay help</code> , <code>/assay intake</code> , <code>/assay frame</code> , <code>/assay spec</code> , <code>/assay discovery</code> , <code>/assay execute</code> , <code>/assay validate</code> , <code>/assay deliver</code> , <code>/assay status</code> , <code>/assay finish</code> , <code>/assay resume</code> , or <code>/assay ledger</code> .

Intake And Framing

Skill	Plain description	Use this when...
<code>stakeholder-requirements-gathering</code>	Turns a loose request into clear needs.	You need to clarify the question, decision, audience, track, or acceptance criteria.
<code>context-packager</code>	Packages goals, systems, files, and constraints.	You need to organize the context Claude should use.
<code>analysis-planning</code>	Structures question, scope, metrics, risks, and deliverables.	You are preparing the Stage 2 spec.
<code>analysis-assumptions-log</code>	Tracks assumptions, trade-offs, risks, and validation plans.	You need an audit trail for choices that could change the answer.
<code>business-metrics-calculator</code>	Defines and calculates standard business metrics.	You need exact metric definitions or common SaaS, e-commerce, finance, marketplace, or product metrics.
<code>schema-mapper</code>	Maps tables, fields, and relationships.	You are exploring unfamiliar data or finding join paths.

Analysis Techniques

Skill	Plain description	Use this when...
<code>programmatic-eda</code>	Runs first-pass data inspection.	You need data shape, distributions, missing values, correlations, or obvious risks.
<code>data-quality-audit</code>	Audits data against rules and source expectations.	You need to check data quality or prepare validation evidence.
<code>ab-test-analysis</code>	Reviews A/B test setup and results.	You need conversion lift, sample split, confidence, guardrails, or rollout advice.
<code>cohort-analysis</code>	Tracks groups over time.	You need retention, revenue, adoption, or behavior by start date, channel, or plan.
<code>funnel-analysis</code>	Measures multi-step conversion and drop-off.	You need signup, checkout, onboarding, renewal, lead, or process funnel analysis.
<code>impact-quantification</code>	Sizes business impact and ROI.	You need opportunity size, cost, savings, or prioritization by value.
<code>root-cause-investigation</code>	Investigates unexpected metric movement.	A metric changed and stakeholders need drivers.
<code>segmentation-analysis</code>	Groups customers, users, products, or accounts.	You need meaningful behavior groups or outcome comparisons by segment.
<code>time-series-analysis</code>	Analyzes trends, seasonality, anomalies, and forecasts.	You need metric movement over time or cautious planning ranges.
<code>sql-to-business-logic</code>	Translates SQL into business rules.	You need to explain or check what a query really counts.
<code>semantic-model-builder</code>	Documents shared metric and entity definitions.	You need a semantic layer for recurring reports or reproducible

Skill	Plain description	Use this when...
		analysis.
<code>dashboard-specification</code>	Designs dashboard requirements.	You need users, decisions, metrics, layout, refresh, access, and validation needs.
<code>visualization-builder</code>	Chooses and designs clear BI visuals.	You need charts, dashboard views, or presentation-ready visuals.

Validation And Review

Skill	Plain description	Use this when...
<code>metric-reconciliation</code>	Compares numbers across sources and explains differences.	Results must tie to finance, BI, warehouse, product, or migrated-system totals.
<code>query-validation</code>	Reviews SQL correctness and business logic.	You need joins, filters, aggregation, source ties, or performance checked.
<code>analysis-qa-checklist</code>	Runs BI quality checks before delivery.	You are preparing validation or checking calculations.
<code>peer-review-template</code>	Runs structured adversarial review and scoring.	You need a red-team review before delivery.

Communication

Skill	Plain description	Use this when...
<code>insight-synthesis</code>	Turns findings into decision-ready insights.	You need conclusions and recommendations from validated results.
<code>executive-summary-generator</code>	Writes concise leadership summaries.	You need a short report for executives.
<code>data-narrative-builder</code>	Builds plain-language reports and commentary.	You need a delivered story from validated analysis.
<code>technical-to-business-translator</code>	Converts technical BI language to business language.	You need SQL, statistics, or methodology explained plainly.
<code>methodology-explainer</code>	Explains methods, assumptions, and limits.	You need to tell stakeholders how the analysis was done.

Documentation

Skill	Plain description	Use this when...
<code>analysis-documentation</code>	Creates reproducible analysis documentation.	You need to archive question, method, data, results, and validation.
<code>data-catalog-entry</code>	Documents datasets, ownership, quality, and lineage.	You need a catalog entry or data dictionary.
<code>analysis-retrospective</code>	Captures lessons after analysis or release.	You need to record what worked, what failed, and what to watch next time.

Production And Output

These are domain skills used heavily when a result becomes a report or dashboard:

Skill	Plain description	Use this when...
<code>dashboard-specification</code>	Defines recurring dashboard behavior.	You are building a data product.
<code>semantic-model-builder</code>	Makes shared definitions reusable.	You need recurring metrics to stay consistent.
<code>visualization-builder</code>	Creates decision-readable charts.	You need a chart, report visual, or dashboard panel.
<code>analysis-documentation</code>	Saves the reproducible trail.	Someone needs to rerun or audit the work.
<code>data-catalog-entry</code>	Records data ownership and meaning.	The report depends on reusable datasets.

How To Ask For A Skill

You usually do not need to name skills. The `/assay` router chooses them from the stage.

You can still ask plainly:

```
Use cohort analysis to compare renewal retention by signup quarter.
```

or:

```
Use metric reconciliation to tie the dashboard total to Finance.
```

Agents And Models

An agent is a focused AI worker. A model is the AI system doing the thinking. The kit uses the main model for judgment and sub-agents (worker agents with narrow tasks) for legwork.

Previous: Skills | [Index](#) | Next: Reports And Dashboards

The Simple Rule

Main model equals judgment.

Sub-agents equal legwork.

Cross-family review equals second opinion.

Cross-family means a different model family reviews the work.

Main Model

The main model stays in the conversation with you.

Use it for:

- framing the request;
- asking you for rulings;
- deciding what to delegate;
- interpreting results;
- accepting or rejecting sub-agent work;
- writing the final answer.

It should not silently decide a methodology fork (choice that changes numbers). It should ask you.

The 4 Sub-Agents

Agent	What it does	What it should not do
eda- profiler	Profiles data shape, freshness, nulls, duplicates, outliers, and risks.	It should not decide material method choices.
query- runner	Runs ruled queries and calculations exactly to the spec.	It should not change the method or write stakeholder delivery copy.
reconciler	Ties results to source-of-truth and records validation evidence.	It should not wave through unreconciled results.
red-teamer	Attacks conclusion, method, reconciliation, score, and wording.	It should not soften blocking issues for convenience.

Nulls are blank or unknown values. Outliers are unusual values that can skew.

Why Use Sub-Agents

Sub-agents do bounded work:

```
Profile this table for missing values and duplicates.
Run this ruled query.
Compare this total to Finance.
Attack this conclusion for weak assumptions.
```

The main model keeps responsibility for judgment:

```
Does this caveat change the answer?
Should we ask the operator for a ruling?
Is this result ready to deliver?
```

The Model Dial

The model dial is the cost-and-quality choice for AI work.

Use the strongest model for:

- methodology (chosen analysis approach);
- risk decisions;
- synthesis;
- final review.

Use cheaper sub-agents for:

- profiling;
- counting;
- running queries;
- drafting bounded notes;
- reconciliation checks.

Use cross-family review when:

- the work is high-stakes;
- the method is debatable;
- the result will drive money, headcount, or strategy;
- you want a second opinion that may catch confirmation bias.

Confirmation bias means seeing only supporting evidence.

What You Will See

During execution, Claude may say:

```
I am sending the data profile to eda-profiler and the ruled calculation  
to query-runner. I will use their outputs to synthesize the answer.
```

During validation, Claude may say:

```
I am sending reconciliation to reconciler and independent review to red-  
teamer. Self-review does not count.
```

That split is intentional. It keeps the person-facing conversation focused on judgment while the workers handle repeatable tasks.

Reports And Dashboards

Deliverables are the files you give to people. The kit can render reports and dashboards after the delivery gates pass.

Previous: Agents And Models | [Index](#) | Next: Recurring And Monitoring

Report Engine

The report engine reads a JSON report contract. JSON means structured key-value data. It always writes a self-contained HTML report, meaning a browser page saved as a file.

Type:

```
bash .claude/workflows/report-render.sh <analysis-id> <deliverable-json>
```

Usually Claude runs this during:

```
/assay deliver <analysis-id>
```

What you will see:

```
assay-report-html:.assay/deliverables/renewal-revenue-q2/report-20260624T153000Z.html
assay-report-pdf:.assay/deliverables/renewal-revenue-q2/report-20260624T153000Z.pdf
assay-report-receipt:.assay/receipts/renewal-revenue-q2-deliverable-receipt.json
```

If no PDF tool is installed:

```
assay-report-pdf-note:No PDF renderer, meaning a tool that makes PDF files, was available. Open the HTML report and use print-to-PDF if a PDF is needed.
```


PDF means print-ready file for sharing. Print-to-PDF means using the browser print dialog to save a PDF file.

Report Contents

The report contract supports:

- title;
- audience;
- conclusion;
- key findings;
- evidence;
- methodology (chosen analysis approach);
- caveats (limits that affect trust);
- reconciliation notes;
- score;
- figures;
- next steps.

The report is self-contained. That means it does not need outside scripts or websites to display.

PDF Requirements

The report engine tries these PDF renderers (tools that make PDF files):

- `pandoc`;
- `wkhtmltopdf`;
- Chrome;
- Chromium.

HTML reports always work if Python 3 is available. PDF is best effort.

If PDF does not generate, open the HTML file in your browser and choose print-to-PDF.

Dashboard Engine

The dashboard engine reads an `assay-dashboard/v1` JSON contract and writes a static HTML dashboard, meaning a browser page saved as a file.

Type:

```
bash .claude/workflows/dashboard-render.sh <analysis-id> <dashboard-json>
```

Usually Claude runs this during:

```
/assay deliver <analysis-id>
```

What you will see:

```
assay-dashboard-html:.assay/deliverables/renewal-scorecard/dashboard-20260624T153000Z.html
assay-dashboard-receipt:.assay/receipts/renewal-scorecard-deliverable-receipt.json
```

Dashboard Panel Types

The dashboard renderer supports:

Type	Meaning	Use it for
<code>kpi</code>	Main number watched for decisions.	Revenue, margin, retention, backlog, risk count.
<code>bar</code>	Comparison across groups.	Segment, owner, location, product, channel.
<code>line</code>	Values tracked over time.	Trends, weekly movement, monthly totals.
<code>table</code>	Detail rows.	Exceptions, owners, follow-up lists.

Charts use inline SVG (browser-drawn chart image format). They render offline without a chart library.

Dashboard Required Fields

The dashboard contract needs:

```
{
  "schemaVersion": "assay-dashboard/v1",
  "analysisId": "renewal-scorecard",
  "title": "Renewal Scorecard",
  "audience": "sales leadership",
  "refreshNote": "Weekly after Monday finance refresh.",
  "panels": []
}
```

`panels` must contain at least one panel.

Branding

Brand settings live in `assay.config.jsonc`:

```
"report": {
  "orgName": "Your Organization",
  "logoPath": "",
  "accentColor": "#2563eb",
  "footer": "Confidential – share only with the approved audience.",
  "outputFormats": ["html", "pdf"]
}
```

What each field does:

- `orgName`: name shown in the report header.
- `logoPath`: local logo image path from the project folder, if any.
- `accentColor`: six-digit hex color, such as `#2563eb`.
- `footer`: confidentiality or sharing note.
- `outputFormats`: requested formats for reports.

Tool-specific exports for Power BI, Tableau, Looker, or Metabase are future work in the current repo. The verified engine today produces universal static HTML.

Recurring And Monitoring

DATA PRODUCT work means a recurring report or dashboard. The kit treats data products more strictly because a wrong recurring number repeats until someone catches it.

Previous: Reports And Dashboards | [Index](#) | Next: Conductor And Help

What Is Verified In This Repo

The current repo verifies these data-product behaviors:

- DATA PRODUCT is a first-class track.
- Stage 4d designs the product.
- Stage 10 monitors refreshes and drift.
- data products require stronger validation and review scoring.
- `reprocheck` warns strongly when a data product has no `reproCommand`.
- the report engine and dashboard engine write per-run metric snapshots.
- `deliverable-diff.sh` compares the current run with the prior run.
- `driftcheck.sh` checks refresh health and metric drift.
- `distribution-manifest.sh` writes a local delivery handoff, or withholds it when sensitive data needs sign-off.
- `metric-catalog.json` and `metric-store.sh` maintain shared metric definitions.

Versioning

Versioning means keeping named versions of outputs. For recurring reports, versioning answers:

- Which report run did we send?
- Which inputs did it use?
- Which metric definitions were active?
- Which caveats applied?
- What changed since the last run?

Operator use:

```
/assay status renewal-scorecard
```

Then check deliverable receipts (saved proof files) under:

```
.assay/receipts/
```

and output files under:

```
.assay/deliverables/<analysis-id>/
```

What Changed Since Last Run

Recurring reports need a clear change summary:

- metric values changed;
- source data refreshed;
- methodology rulings changed;
- dashboard layout changed;
- caveats changed;
- validation status changed.

`deliverable-diff` means a comparison between report versions. The script writes:

- `diff-<timestamp>.txt`: plain-language change notes;
- `latest.json`: pointer to the newest snapshot.

The exact command is:

```
bash .claude/workflows/deliverable-diff.sh <analysis-id> <metrics-  
snapshot-json> <artifact-path> [assay.config.jsonc]
```

During report or dashboard delivery, the renderer runs this automatically after it writes the new metrics snapshot.

Drift And Refresh Monitoring

Drift means numbers move unexpectedly. Refresh means new data arrives for the report.

The exact command is:

```
bash .claude/workflows/driftcheck.sh <analysis-id> <metrics-snapshot-
json> [assay.config.jsonc]
```

`driftcheck.sh` writes:

- `drift-<timestamp>.txt`: the plain-language monitoring result;
- `latest-drift.json`: structured drift state for later tools.

The block-vs-warn rule is:

- broken or empty data-product refresh BLOCKS delivery;
- metric drift WARNS, because a changed number may still be real.

Broken refresh means `refreshOk` is false or `rendererStatus` is failed or error. Empty refresh means `rowCount` is zero or below. Those block only when the spec receipt says the track is `data-product`.

Metric drift compares the new snapshot to the previous `latest.json` snapshot. The config key is:

```
"monitoring": {
  "defaultTolerance": 0.10,
  "metrics": {
    "net_retention": { "tolerance": 0.05, "mode": "relative" },
    "open_invoice_count": { "tolerance": 10, "mode": "absolute" }
  }
}
```

Relative tolerance means percent movement from the prior run. Absolute tolerance means raw number movement.

Distribution Handoff

Distribution means who receives the report and where it goes. A distribution manifest means a delivery handoff record.

The exact command is:

```
bash .claude/workflows/distribution-manifest.sh <analysis-id> <artifact-path> <timestamp> <metrics-snapshot-json> [assay.config.jsonc]
```

The manifest records:

- audience;
- channel description;
- cadence;
- data classification;
- data-safety receipt path, when present;
- local send status.

The config key is:

```
"distribution": {  
  "audience": "finance leaders",  
  "channelDescription": "email to finance-leaders",  
  "cadence": "monthly after finance close"  
}
```

Sensitive-data distribution is withheld when sensitive data lacks data-safety sign-off. Withheld means no distribution manifest is written. Sensitive data means personal identifying info, health info, payroll, or customer records.

For sensitive data, `datacheck` requires:

- delivery audience;
- whether data leaves the company;
- export destination;
- row-level or aggregate detail;

- operator sign-off.

Living Metric Store

A living metric store means maintained metric definitions. It prevents “renewal revenue” from changing meaning across runs.

The shared file is:

```
metric-catalog.json
```

The path comes from `metricCatalogPath` in `assay.config.jsonc`. If that is not set, the default is `metric-catalog.json`.

`metric-store.sh` manages the catalog:

```
bash .claude/workflows/metric-store.sh add <name> <definition>
<sourceOfTruth> <owner> <format> [notes]
bash .claude/workflows/metric-store.sh get <name>
bash .claude/workflows/metric-store.sh list
bash .claude/workflows/metric-store.sh check <name> <definition>
```

What each command does:

- `add`: creates or updates one metric definition.
- `get`: prints one metric definition.
- `list`: prints the known metrics.
- `check`: compares a proposed definition to the catalog.

During `/assay spec`, Claude checks each proposed metric against the catalog:

- `metric-store:match`: use the catalog definition.
- `metric-store:not-found`: ask whether to add the metric before continuing.
- `metric-store:differs`: escalate as drift, meaning definitions split across analyses.

The catalog is a shared definition record. It does not replace validation against source systems.

Recurring Report Checklist

Before a data product is delivered, confirm:

- the track is `data-product`;
- source-of-truth is configured;
- refresh cadence is ruled and recorded;
- the dashboard or report has an owner;
- sensitive-data handling is recorded;
- `reproCommand` is set or the warning is accepted;
- validation and adversarial review receipts exist;
- metric definitions are checked against `metric-catalog.json`;
- distribution handoff is clear, or sensitive-data withholding is expected.

Conductor And Help

The conductor is the part of the kit that keeps the current analysis and next step visible. It uses the active pointer, status helper, help helper, and reminder hook.

Previous: Recurring And Monitoring | [Index](#) | Next: Troubleshooting FAQ

Active Analysis

Active analysis means the saved work to resume first. The kit stores it here:

```
.assay/active.json
```

When you frame work, Claude sets it:

```
assay-active-set:renewal-revenue-q2:analysis
```

When delivery succeeds, Claude clears it:

```
assay-active-clear:renewal-revenue-q2
```

If delivery fails, it does not clear the active pointer. That helps you resume later.

Every-Turn Reminder

The installer adds a `UserPromptSubmit` hook. A hook is a small command that runs at a set time. This hook prints the active analysis and next step each turn.

It is a reminder, not the hard gate. The hard blocks are the gate scripts, such as `questioncheck` and `validationcheck`.

/assay help

Type:

```
/assay help
```

What you will see:

```
assay help
```

```
What this kit does: it guides BI work from question to trusted answer.
```

```
...
```

```
Current next step:
```

```
next required step: /assay intake
```

```
No active analysis or receipts were found. Receipts are saved proof files.
```

If an active analysis exists, help uses it:

```
Current next step:
```

```
next required step: /assay validate renewal-revenue-q2
```

/assay status

Type:

```
/assay status
```

If there are multiple in-flight analyses, you will see one line per analysis:

```
renewal-revenue-q2    next: /assay validate renewal-revenue-q2
```

```
blocker: missing-validation
```

```
labor-scorecard      next: /assay deliver labor-scorecard
```

```
blocker: none
```

For one analysis:

```
/assay status renewal-revenue-q2
```

You will see:

```
assay-state: renewal-revenue-q2
completed stages:
  - Stage 2 spec receipt
  - governing-doc baseline
  - Stage 5 discovery record
  - methodology rulings
open findings:
  - missing-validation [validationcheck]: Missing Stage 7 validation
receipt...
stage: Stage 7 Validate
blocking gate: validationcheck (missing-validation)
next required step: /assay validate renewal-revenue-q2
```

/assay finish

Finish means continue a stalled analysis from saved proof.

Type:

```
/assay finish renewal-revenue-q2
```

The helper reports the next required step. Claude must continue from that step only. It must not recompute completed stages or jump around a gate.

/assay resume

Resume is an alias for finish. Alias means another command name for the same action.

If you remember the id:

```
/assay resume renewal-revenue-q2
```

If you do not remember the id:

```
/assay resume
```

If there is no active analysis, you will see:

```
assay-state: no active analysis – run /assay status or /assay help
```

Then type:

```
/assay status
```

/assay ledger

The ledger is the saved list of methodology rulings. Methodology rulings are approved choices that can change numbers.

Type:

```
/assay ledger
```

Claude runs:

```
bash .claude/workflows/decision-ledger.sh list
```

You can ask:

```
Show the rulings for renewal-revenue-q2.
```

or:

```
Show prior refresh-cadence rulings.
```

New User Guidance

The installed `CLAUDE.md` tells Claude to proactively orient new or confused operators. If you ask a data question without `/assay`, Claude should briefly explain the loop and start with `/assay intake` or `/assay frame`.

You can always say:

```
I am new. Please tell me the next command and why it matters.
```

Troubleshooting FAQ

This page covers common beginner problems, especially if you are new to AI tools and terminal-driven work.

Previous: Conductor And Help | [Index](#) | Next: Glossary

It stopped and asked for a spec. Did I do something wrong?

No. The kit is protecting the analysis.

You may see:

```
assay-gate-failed:missing-spec  
questioncheck: Stage 6 is blocked because no Stage 2 spec receipt  
exists...
```

Spec means written question, metrics, scope, and valid answer. The fix is:

```
/assay spec <analysis-id>
```

Tell Claude:

```
Here is the question, metric definition, scope, valid answer, and  
decision impact.
```

What is a receipt?

A receipt is a saved proof file. It records that a step happened and what proof exists.

Receipts live under:

```
.assay/receipts/
```

You usually do not edit them. Claude writes them through the kit.

The PDF did not generate. Is the report lost?

No. HTML still works.

You may see:

```
assay-report-pdf-note:No PDF renderer, meaning a tool that makes PDF
files, was available. Open the HTML report and use print-to-PDF if a PDF
is needed.
```

Open the HTML report path in your browser and choose print-to-PDF.

Optional PDF tools are:

- `pandoc`;
- `wkhtmltopdf`;
- Chrome;
- Chromium.

I closed the window. How do I pick up?

Open the project folder again. Start Claude Code:

```
claude
```

Then type:

```
/assay resume
```

If that does not find an active analysis:

```
/assay status
```


Then resume the analysis id shown:

```
/assay resume <analysis-id>
```

It says source-of-truth is not configured.

Source-of-truth means official place to compare. High-stakes work and data products need it configured.

You may see:

```
assay-gate-failed:source-of-truth-unconfigured
```

Fix:

```
/assay intake
```

Tell Claude:

```
For renewal revenue, source-of-truth is the Finance close report.  
For gross margin, source-of-truth is NetSuite.  
For active customers, source-of-truth is Salesforce account status.
```

The numbers need a source of truth, but we have two.

Say that plainly. This is a methodology fork (choice that changes numbers).

Example:

```
Finance has the close report, but Sales uses Salesforce. Finance is  
official for total renewal revenue. Salesforce is useful for owner and  
segment cuts.
```

Claude should ask you to rule which source wins for each metric.

It found a new methodology fork during execution.

That is expected sometimes. It means a material choice appeared after data inspection.

Example:

```
Customer segment differs between Finance and Sales. This can change the
driver table.
```

Answer with a ruling:

```
Use Finance segment for the official result. Add Sales segment as a
supporting view only.
```

It says data classification is unknown.

Classification means how sensitive the data is.

Fix by telling Claude:

```
Data classification is internal.
```

or:

```
Data classification is customer records. Delivery audience is internal
finance leadership. Data leaves company: false. Detail level: aggregate.
I approve this handling.
```

It says a governing doc changed.

Governing docs are protected rule files. The kit snapshots them at discovery and blocks if they change before delivery.

If the change was accidental, undo it.

If a person approved the change:

```
ASSAY_GOVCHECK_APPROVED=1 bash .claude/workflows/govcheck.sh resnapshot
<analysis-id>
```

Do not run that just to get around the gate.

It says reproducibility is unverified.

Reproducibility means re-running work gets the same answer.

If `reproCommand` is unset, the gate warns but passes:

```
reprocheck: NOTE – reproducibility unverified because reproCommand is
unset.
assay-gate-ok: reprocheck
```

For recurring reports, add a command in `assay.config.jsonc`:

```
"reproCommand": "bash scripts/build-report.sh"
```

Claude is asking too many questions.

For BI work, questions are often the safeguard. Answer briefly:

```
Use invoice date because Finance reports by invoice month.
```

If a question does not matter, say:

```
This choice does not change the number or decision. Use the simpler
option and record it as a low-risk assumption.
```

Claude is making assumptions.

Stop it and ask for the assumption log:

List every assumption you are making, what could change if it is wrong, and which ones need my ruling.

Tips For Working With An AI Assistant

Be specific:

Compare Q2 to Q1, using invoice date, excluding one-time services.

Let it ask:

Ask me before choosing any method that changes the number.

Review its work:

Show the reconciliation and caveats before delivery.

Name official sources:

Finance close report is official for revenue. Tableau is not official for revenue totals.

Correct it directly:

That definition is wrong. Renewal revenue excludes new-logo ARR.

Glossary

Plain definitions for terms used in this guide and the toolkit.

Previous: Troubleshooting FAQ | [Index](#)

A

A/B test: comparison between two versions.

Absolute tolerance: allowed raw-number movement.

Acceptance criteria: conditions required before work is done.

Active analysis: saved work to resume first.

Adversarial review: independent attack on weak spots.

Aggregate: grouped summary, not individual rows.

AI: software that drafts and checks work.

Alias: another command name for same action.

Analysis: one-time answer using data.

Analysis id: short saved-work name.

Anomaly: unexpected value needing review.

ARR: annual recurring revenue.

Assay loop: staged path to trusted analysis.

Assumption: treated as true without full proof.

Audit trail: saved evidence of decisions and checks.

B

Baseline: normal comparison point.

BI: business reporting and metrics.

Block: required stop before continuing.

Business logic: rules that define business meaning.

C

Caveat: limit that affects trust.

Chart library: code used to draw charts.

Claude Code: AI assistant run in a folder.

Cohort: group tracked over time.

Command: text the computer runs.

Confidence: how sure the answer is right.

Confidence interval: likely range around estimate.

Confirmation bias: seeing only supporting evidence.

Contract: required shape for input data.

Conversion: people reaching the next step.

Correlation: two measures moving together.

Cross-family review: different model family checks work.

CSV: plain spreadsheet-style text file.

D

Dashboard: recurring view of business metrics.

Data classification: how sensitive the data is.

Data completeness: how much needed data exists.

Data product: recurring report or dashboard.

Data quality: fitness of data for use.

Decision impact: business choice the answer affects.

Decision ledger: saved list of ruled choices.

Deliverable: file or answer given to people.

Distribution: who receives report and where.

Distribution manifest: delivery handoff record.

Drift: unexpected metric movement.

Driftcheck: recurring metric movement check.

E

EDA: first-pass data inspection.

Eligibility rule: who or what is included.

Evidence: proof supporting a conclusion.

Execute: run ruled analysis or build output.

Export: data leaves analysis workspace.

F

Fail-closed: block until proof exists.

Filter: rule excluding some records.

Forecast: future estimate with uncertainty.

Fork: choice that changes numbers.

Frame: name decision and work track.

Fresh agent: independent worker not previously involved.

G

Gate: required stop-check before continuing.

Git: file version tracking tool.

Governing docs: protected project rule files.

Grain: what one row represents.

Guardrail metric: must-not-get-worse measure.

H

High-stakes: drives money, headcount, or strategy.

HTML: browser page file format.

I

Impact quantification: sizing business effect.

In-flight analysis: saved unfinished analysis work.

Inline definition: short explanation beside term.

Intake: capture project tools and rules.

Internal data: company-only business data.

IQR: middle-half spread.

J

Join: matching records across tables.

JSON: structured key-value data.

JSONC: JSON with comments.

K

KPI: main number watched for decisions.

L

Ledger: saved history of decisions.

Line chart: chart showing values over time.

Lineage: where data comes from.

Living metric store: maintained metric definitions.

M

MAD: typical distance from middle.

Main model: AI leading judgment.

MAPE: average percent miss.

Mean: average value.

Median: middle value, ignores outliers.

Metric: number used for decisions.

Metric catalog: shared metric definition file.

Metric definition: exact calculation rule.

Metric store: maintained metric definition workflow.

Methodology: chosen analysis approach.

Methodology fork: choice that changes numbers.

Methodology ruling: approved method choice.

Model: AI system doing thinking.

Model dial: cost-and-quality model choice.

N

Non-zero: command ended with failure.

Null: blank or unknown value.

O

Operator: person guiding the analysis.

Outlier: unusual value that can skew.

P

p-value: chance result is noise.

Panel: dashboard section.

PATH: command lookup list.

PDF: print-ready file for sharing.

PHI: health information.

PII: personal identifying information.

Plain language: wording a business reader understands.

Power: chance to detect real effect.

Preflight: gate check before chokepoint.

Print-to-PDF: browser saves page as PDF.

Project folder: folder for one work area.

Proxy metric: stand-in measure.

Q

Query: request for data.

Query language: how data is requested.

R

Receipt: saved proof file.

Reconcile: match or explain differences.

Reconciliation: source comparison with explained variance.

Refresh: new data arrives.

Relative tolerance: allowed percent movement.

Refresh cadence: how often data updates.

ReproCommand: command rerunning the work.

Reprocheck: reproducibility delivery gate.

Reproducibility: rerun gets same answer.

Residual: leftover movement after patterns.

Retention: customers or users staying.

ROI: return compared with investment.

Row-level detail: individual records are shown.

Ruling: operator-approved method choice.

S

Sample size: number of records or units.

Schema: map of tables and columns.

Score threshold: minimum allowed review score.

Seasonality: repeating calendar pattern.

Segment: group with shared traits.

Semantic layer: shared metric definitions.

Sensitive data: personal, health, payroll, customer data.

Shell script: small command program.

Significance level: accepted false-win chance.

Source-of-truth: official place to compare.

Spec: written question and metric rules.

Spec receipt: proof the spec exists.

SQL: language for database queries.

Stakeholder: person affected by work.

Static HTML: browser page saved as file.

Sub-agent: worker agent with narrow task.

T

Tableau: BI dashboard tool.

Technical term: specialized word needing definition.

Terminal: text command window.

Threshold: minimum or maximum allowed value.

Time series: values measured in time order.

Track: analysis or data-product path.

Trend: longer-term direction.

Tripwire: rule that forces escalation.

U

Uncertainty: amount we may be wrong.

V

Valid answer: complete answer by agreed rule.

Validation: checking result before delivery.

Validation receipt: proof checks were done.

Variance: difference from expected value.

Versioning: keeping named output versions.

W

Warehouse: central store for analysis data.

Workflow: ordered steps for work.

Z

Zero exit: command ended successfully.